

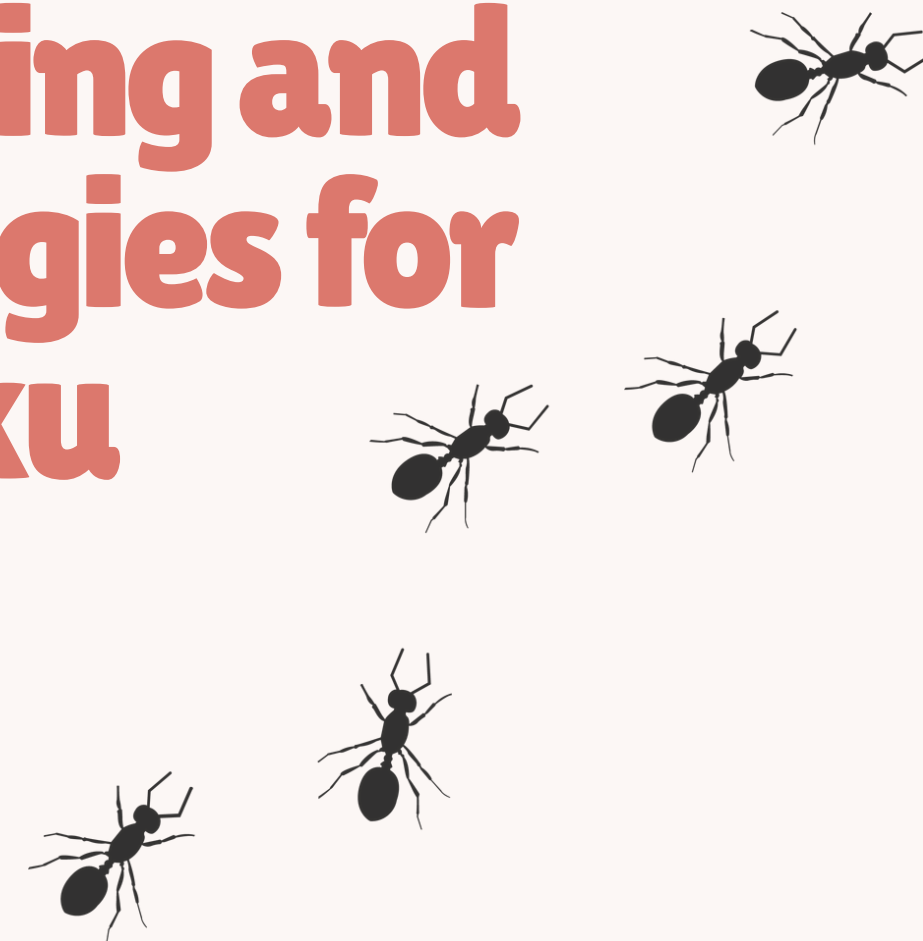


A Multithreaded Multi-Colony Ant Colony Optimization Framework with Ring and Random Communication Topologies for Solving Large-Scale Sudoku

John Michael E. Jabuen*

Gian Myrl D. Renomeron

John Paul T. Yusiong



Outline

1 - Introduction

2 - Related Works

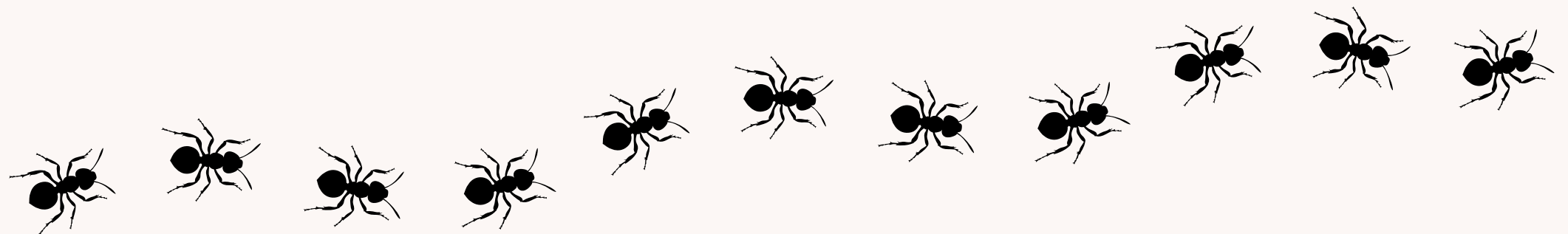
3 - Research Gap

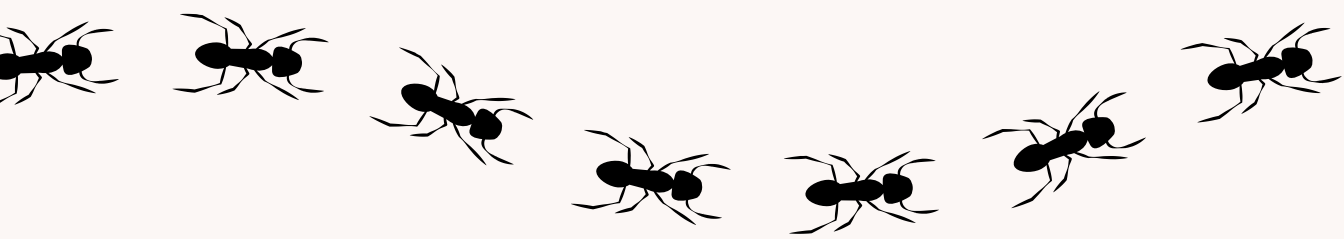
4 - Objectives

5 - Methodology

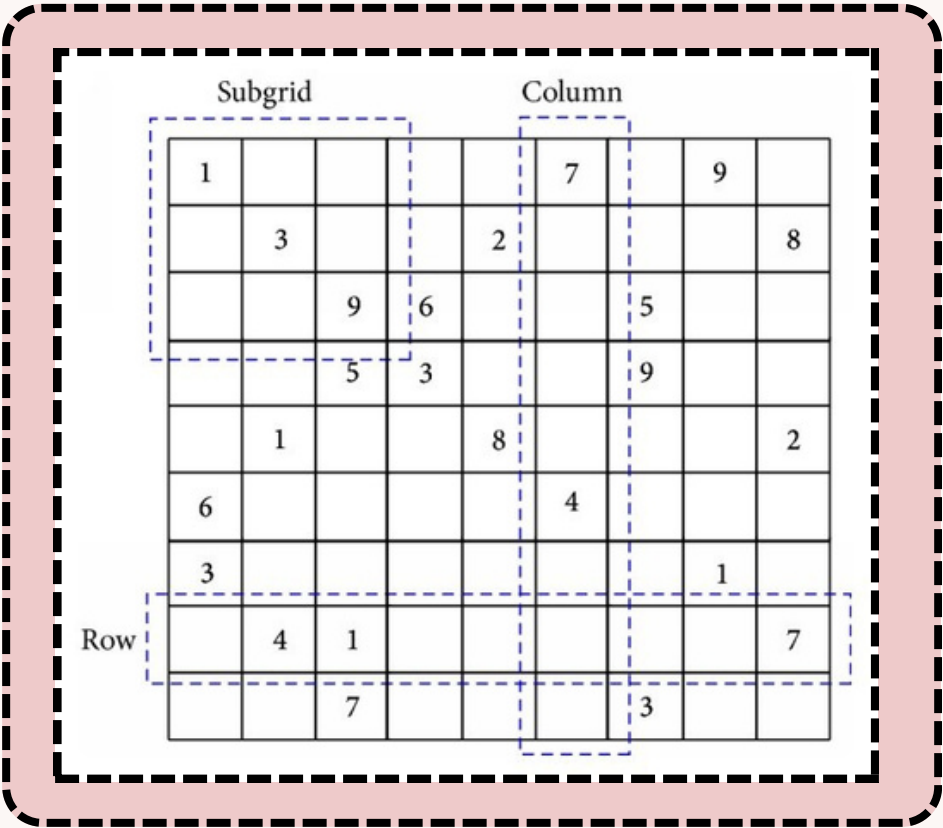
6 - Results and Discussion

7 - Conclusion and Recommendation





Introduction

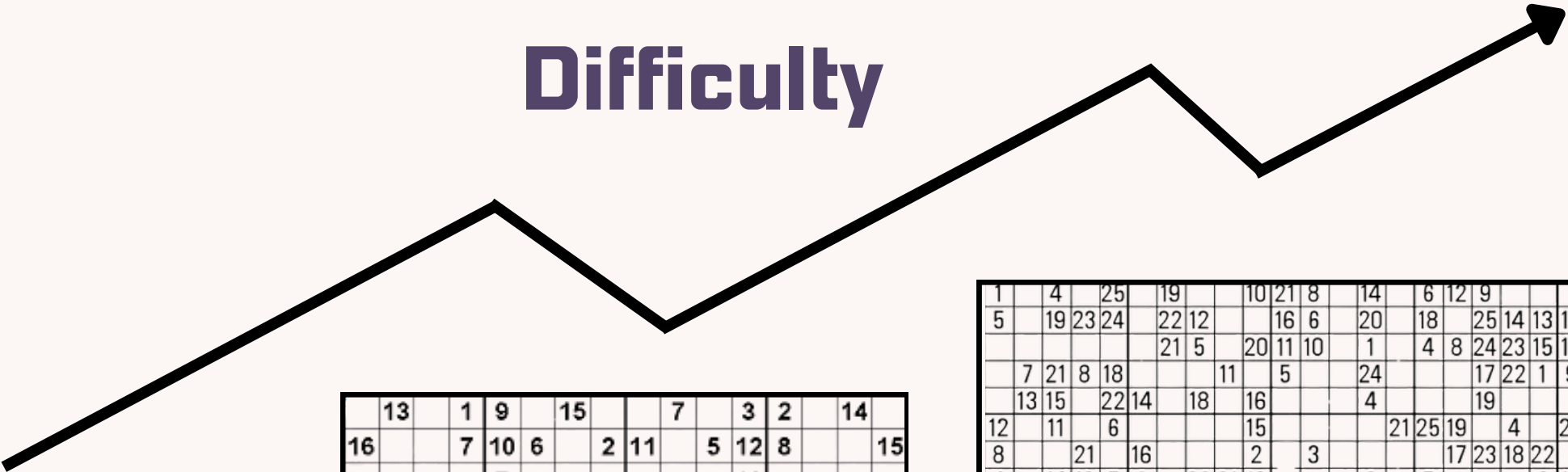


Rule:

Put all numbers from 1 to n without repetition to **row**, **column**, and **subgrid**.



Difficulty



4			9	8	6
			8		
3		7			9
9	4		3	1	
5	1		2		3
	1	8			
			7	5	1
2	5	1	3		7

9 x 9

	13		1	9		15		7		3	2		14	
16			7	10	6		2	11		5	12	8		15
				7							13			
5	11												10	4
1	9	12				13		3				5	6	10
	15				14	11		10	16				12	
13				2	7					8	4			1
	2												4	
	6												5	
12				16	8					9	15			14
	10				4	12		11	6				7	
2	3	15				1		14				6	13	9
6	12												9	5
				1						16				
4			8	15	9		3	1		13	7	11		16
	14		2	5		4		15		10	3		1	

16 x 16

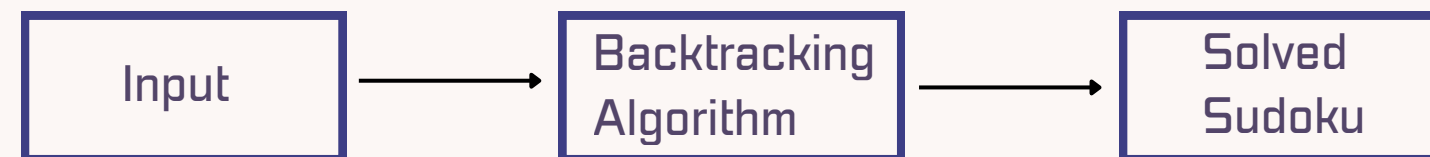
1	4	25	19			10	21	8	14	6	12	9							5
5	19	23	24	22	12			16	6	20	18	25	14	13	10	11		1	15
				21	5	20	11	10	1	4	8	24	23	15	18		16	22	19
	7	21	8	18			11	5	24			17	22	1	9	6	25		
	13	15	22	14	18	16			4			19			24	20	21	1	
12	11	6				15				21	25	19	4	22	14		20		
8		21	16			2	3				17	23	18	22			24	6	
4	14	18	7	9	22	21	19		2	5			6	16	15		11	12	
22	24	23		11	7			4	14	2	12		8	5	19		25	9	
20		5				17	9	12	18	1		7	24				13	4	
13		5	2	23	14	4	18	22	17		20	1	9	21	12		8	11	
14	23	24							20	25	3	4	13	11	21	9	5	18	22
7		11	17	20	24			3	4	1	12		6	14	5	25	13		
	16	9	17	11	7	10	25			13	6		18		19	4		20	
6	15	19	4	13		5		18	11		9	8	22	16	25	10	7		
		2		10	19	3	1	22	9	4	11	15	20		8	23		25	
	24	8	13	1		4	20	17	14		18	16	22	5	11	10			
23	10					18	6	16			17	1	13		3	19	12		
25	5	14	11	17	8	24	13	19	23	15	9		12	20		22		7	
	17	4	22	15	23	11	12	25			18	8	7			14	13		
19	6	23	22	8		1	25	4	14	2	3	7	13	10	11	16			
	4	17		3	24	8	20	23	11	10	25	22		12	13	2	18	6	
	7	16		6	17	2	21	18			19		8				4		
18	9	25	1	2	11		13	22	4	21	5	23	7		15	3		8	
	21	10		12	20	16	19				15	14	4	2	18	23	25	11	7

25 x 25

Related Works

Backtracking

fills empty cells one by one, returning to previous cells if the current path leads to an invalid solution.

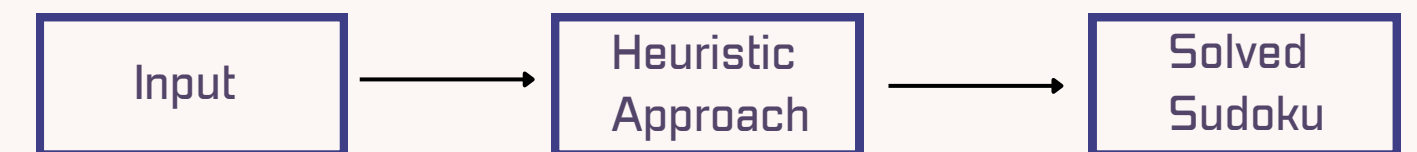


Bhattarai, A., Uprety, D., Pathak, P., Shrestha, S.N., Narkarmi, S. and Sigdel, S., (2025)

Schottlender, M. (2014)

Heuristic

applies logical rules to eliminate some candidates.



Crook, J.F. (2009)

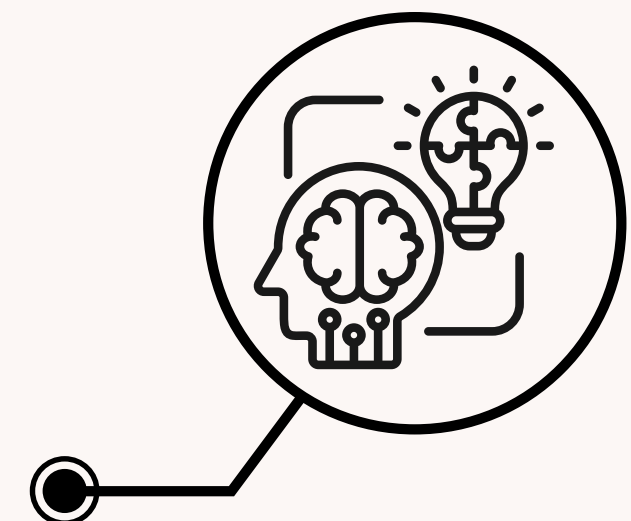
Simonis, H. (2005)

Swarm-based Algorithm

explores the solution space stochastically.



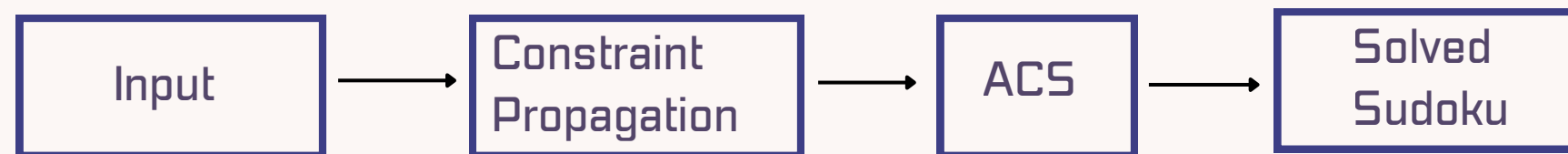
Pacurib, J.A., Seno, G.M.M., and Yusiong, J.P.T, McGerty, S. (2009)
(2009)



Related Works

CP-ACS

Uses CP to remove invalid assignments and ACS for exploring.



H. Lloyd and M. Amos (2020)

DCM-ACO

multiple ant colonies collaborate by sharing pheromone information.

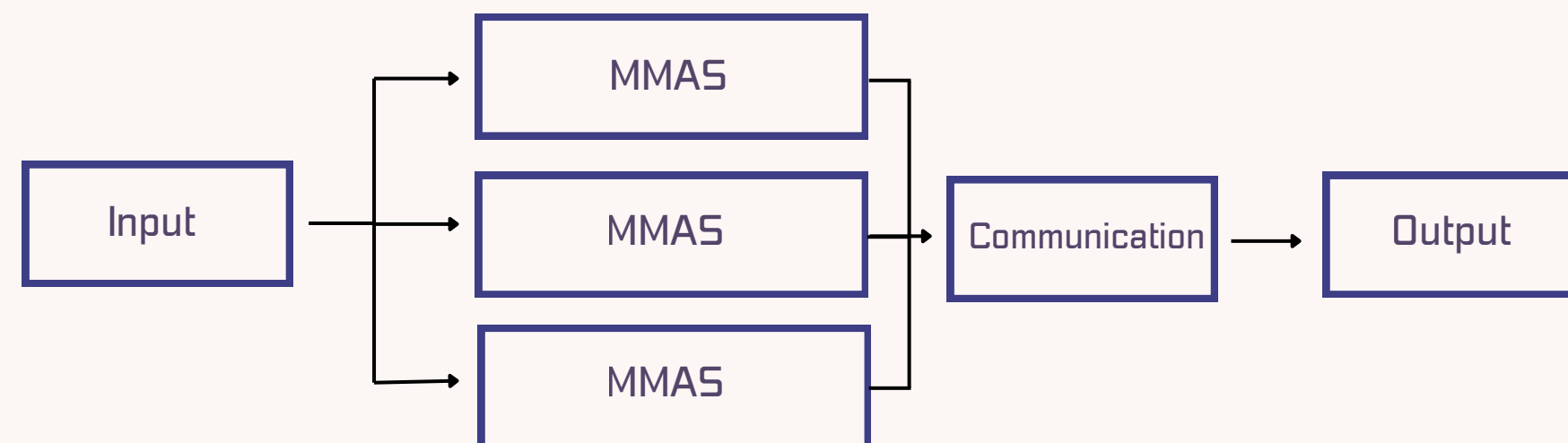


Mo, et al. (2022)

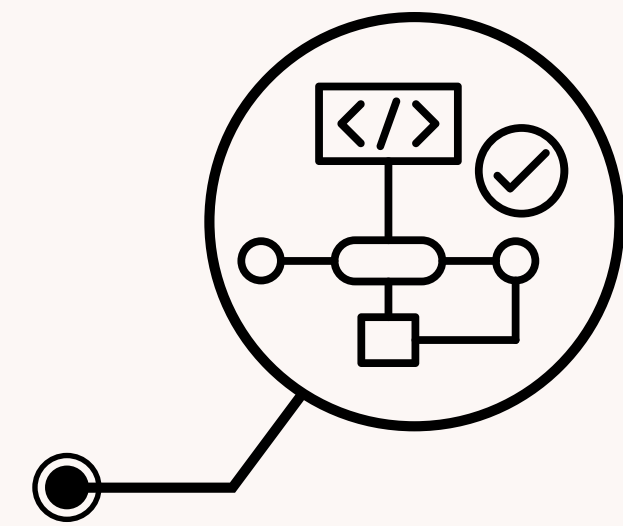
RMACO

Each processor has one Min-Max Ant System colony and shares solutions.

(Randomly Matched Parallel Ant Colony Optimization)



Yang, Q. and Fang, L. (2015)

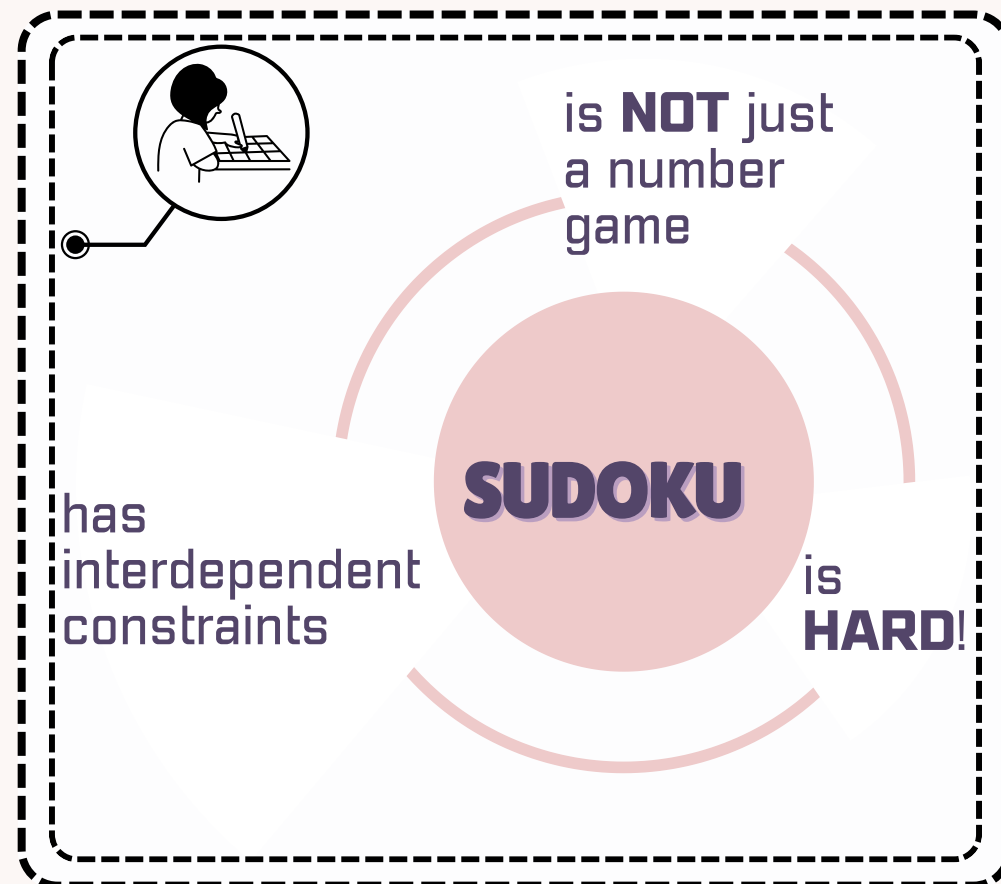


Research Gap

CP-ACS

DCM-ACO

RMACO



Single-threaded solver, no multi-colony collaboration

Heterogeneous multi-colony, but single-threaded

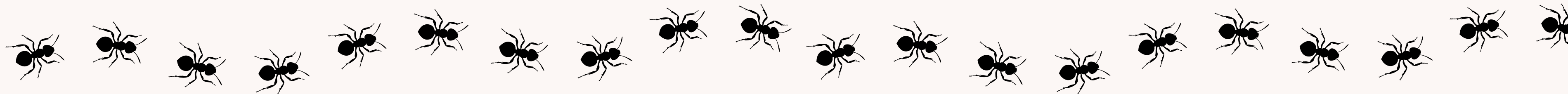
Parallel communication but homogeneous colonies / different domain



No known multithreaded, multi-colony solver exists for large Sudoku.

Objectives

- 1** Implement multiple multi-colony ant optimization (CP-DCM-ACO) solvers running concurrently in separate threads with ring and random communication topologies.
- 2** Compare the proposed solver with the baseline solvers in terms of success rate, solution time, and number of iterations.



Methodology

Sudoku Instances

10 instances in each puzzle size are run to test the robustness and reliability of the solver.

4				9		8		6
					8			
3			7					9
9		4						
				3			1	
5		1			2			3
	1		8					
					7		5	1
2		5		1		3		7

9 x 9

	13		1	9		15		7		3	2		14	
16			7	10	6		2	11		5	12	8		15
				7							13			
5	11												10	4
1	9	12				13		3					5	6
	15				14	11		10	16					12
13				2	7				8	4				1
	2												4	
	6													5
12				16	8				9	15				14
	10				4	12		11	6					7
2	3	15				1		14				6	13	9
6	12												9	5
				1						16				
4			8	15	9		3	1		13	7	11		16
	14		2	5		4			15	10	3		1	

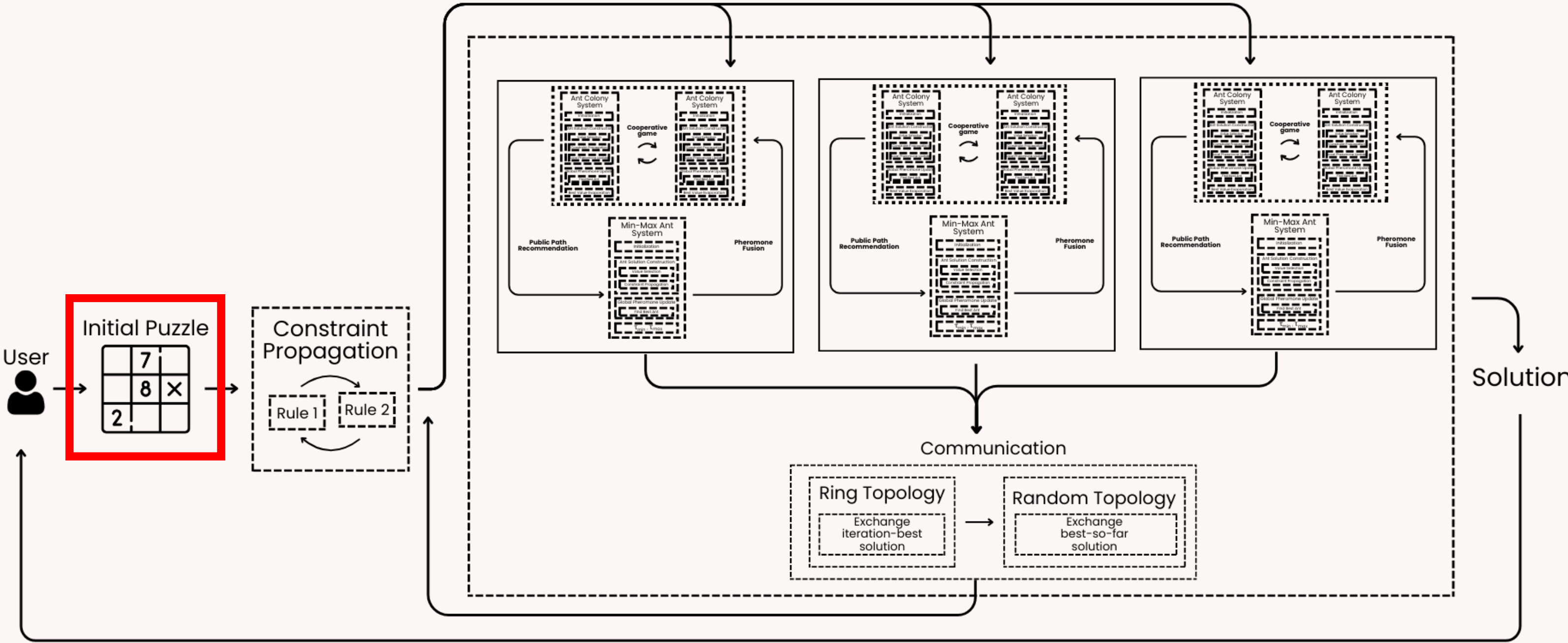
16 x 16

1		4		25		19			10	21	8		14		6	12	9								5
5			19	23	24		22	12			16	6		20		18		25	14	13	10	11		1	15
							21	5		20	11	10		1		4	8	24	23	15	18		16	22	19
	7	21	8	18					11		5			24					17	22	1	9	6	25	
	13	15		22	14		18		16					4				19				24	20	21	17
12		11		6					15							21	25	19		4		22	14		20
8			21		16				2		3					17	23	18	22					24	6
4		14	18	7	9		22	21	19					2		5				6	16	15		11	12
22		24		23			11		7			4		14			2	12		8	5	19		25	9
20				5					17	9		12	18		1			7	24					13	4
13			5		2	23	14	4	18	22		17			20		1	9	21	12				8	11
14	23		24										20	25		3	4	13		11	21	9	5	18	22
7			11	17	20	24					3	4	1	12			6	14		5	25	13			
		16	9		17	11	7	10	25						13	6			18			19	4		20
6	15		19	4	13			5		18	11				9	8	22	16	25	10	7				
			2				10	19	3		1		22	9	4	11	15		20			8	23		25
	24	8	13	1			4	20		17	14			18		16	22	5		11			10		
23	10							18		6		16			17	1		13			3	19	12		
25	5		14	11		17		8	24	13		19	23	15	9			12		20		22		7	
		17	4		22	15		23	11	12	25					18	8		7			14		13	
19	6	23	22	8			1	25	4	14	2		3	7	13	10	11	16							
	4		17		3		24		8	20	23	11	10	25	22			12	13		2	18	6		
		7	16			6	17	2	21		18				19			8						4	
18	9	25	1	2	11			13	22	4		21		5		23	7			15		3		8	
	21	10			12		20	16		19					15	14	4	2	18	23	25	11	7		

25 x 25

DATASET

Methodology

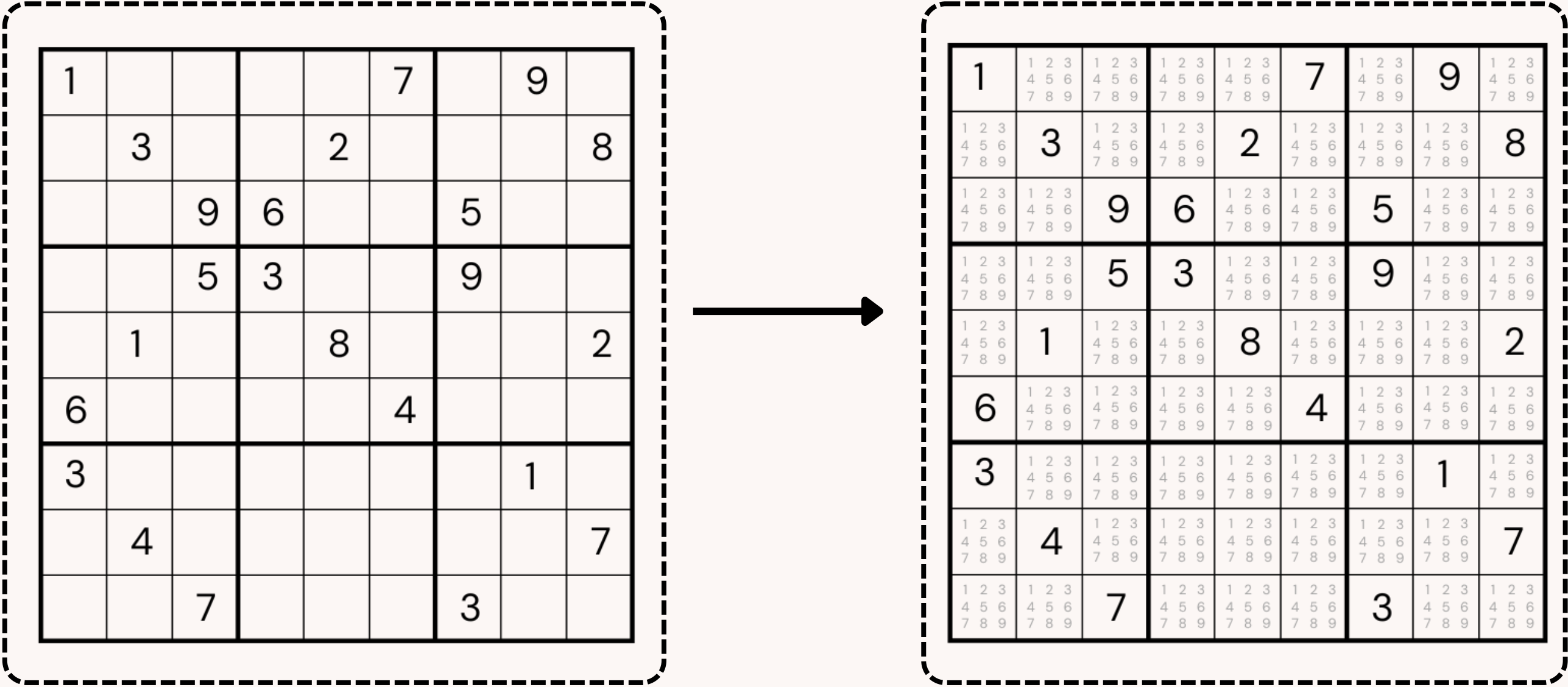
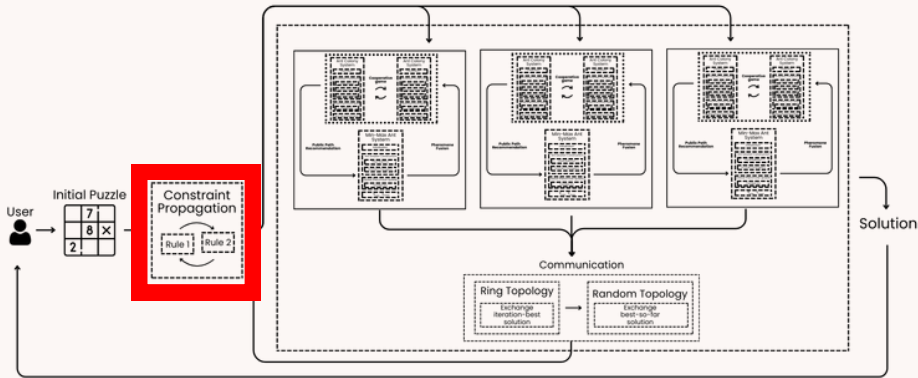


GENERAL OVERVIEW

Methodology

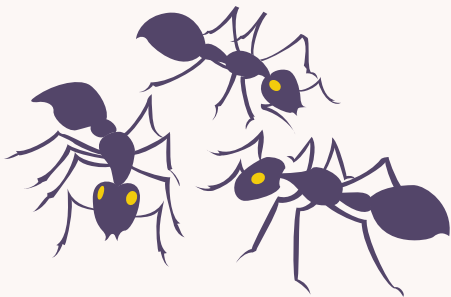
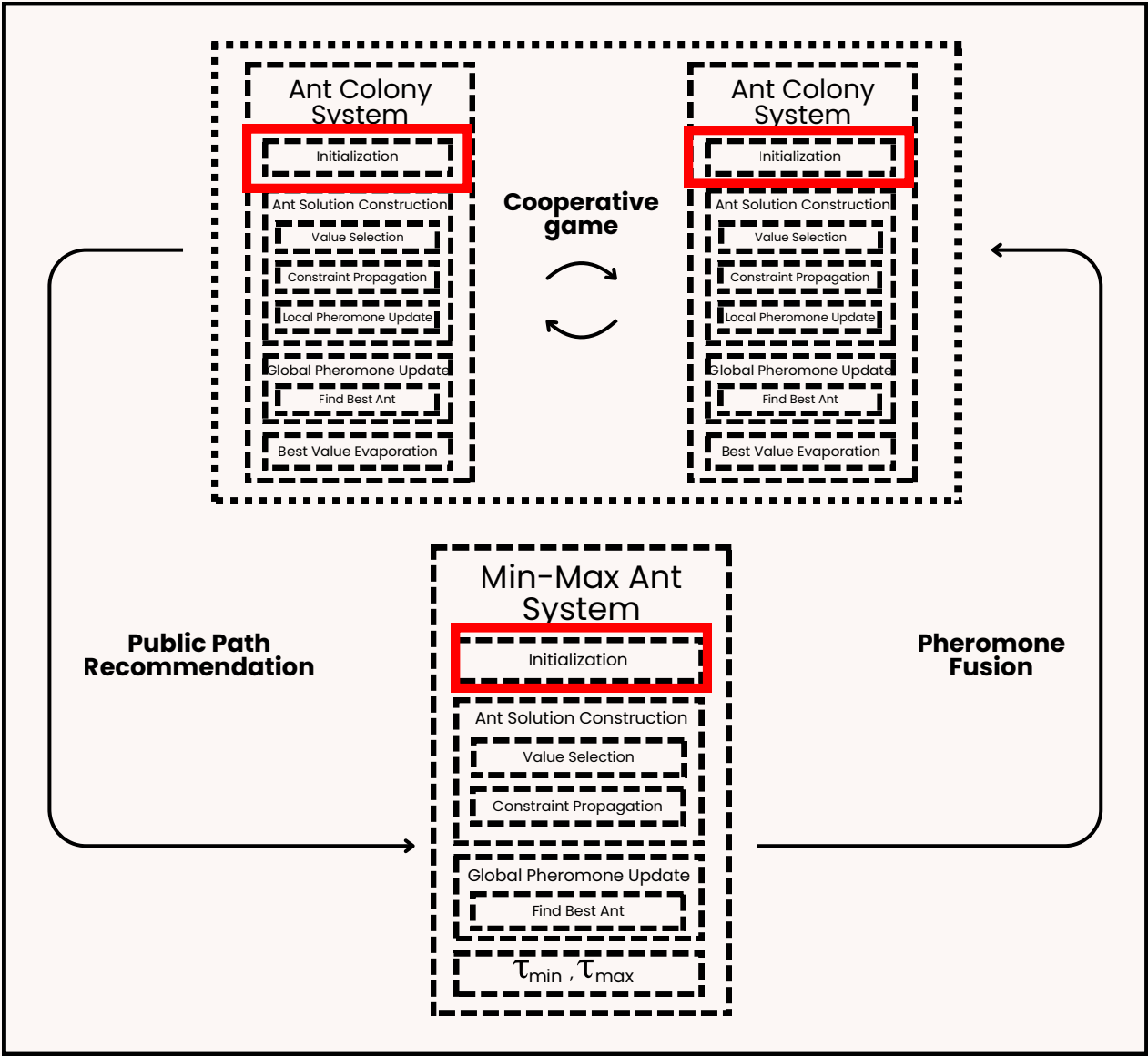
Constraint Propagation

- Rule 1
- Rule 2



Value Set Representation for AI Escargot

Methodology

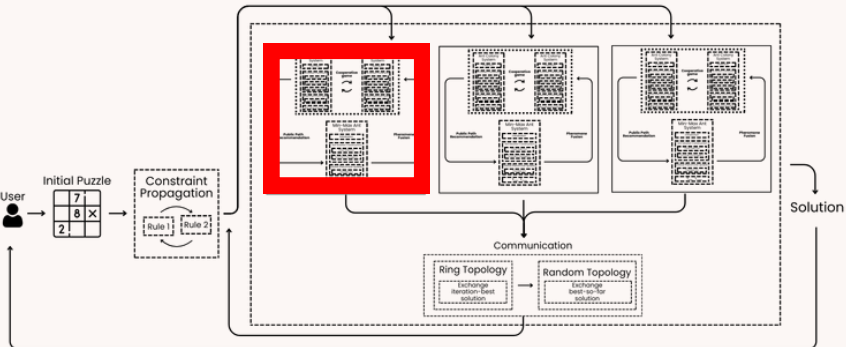


Ant Colony System (ACS)



Min-Max Ant System (MMAS)

- Pheromone Matrix
- Number of Ants
- Puzzle From CP



Methodology

MMAS

Selection Rule

100% Random

Local
Pheromone
Update

No

Pheromone
Limits

Bounded

$[\tau_{\min}, \tau_{\max}]$

ACS

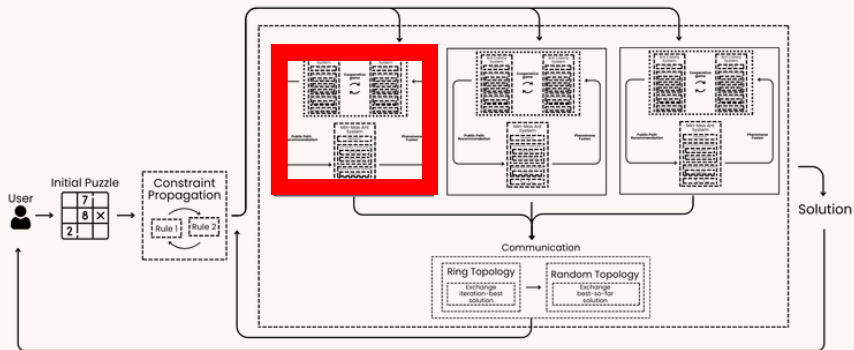
90% Greedy, 10% Random

Yes

Unbounded

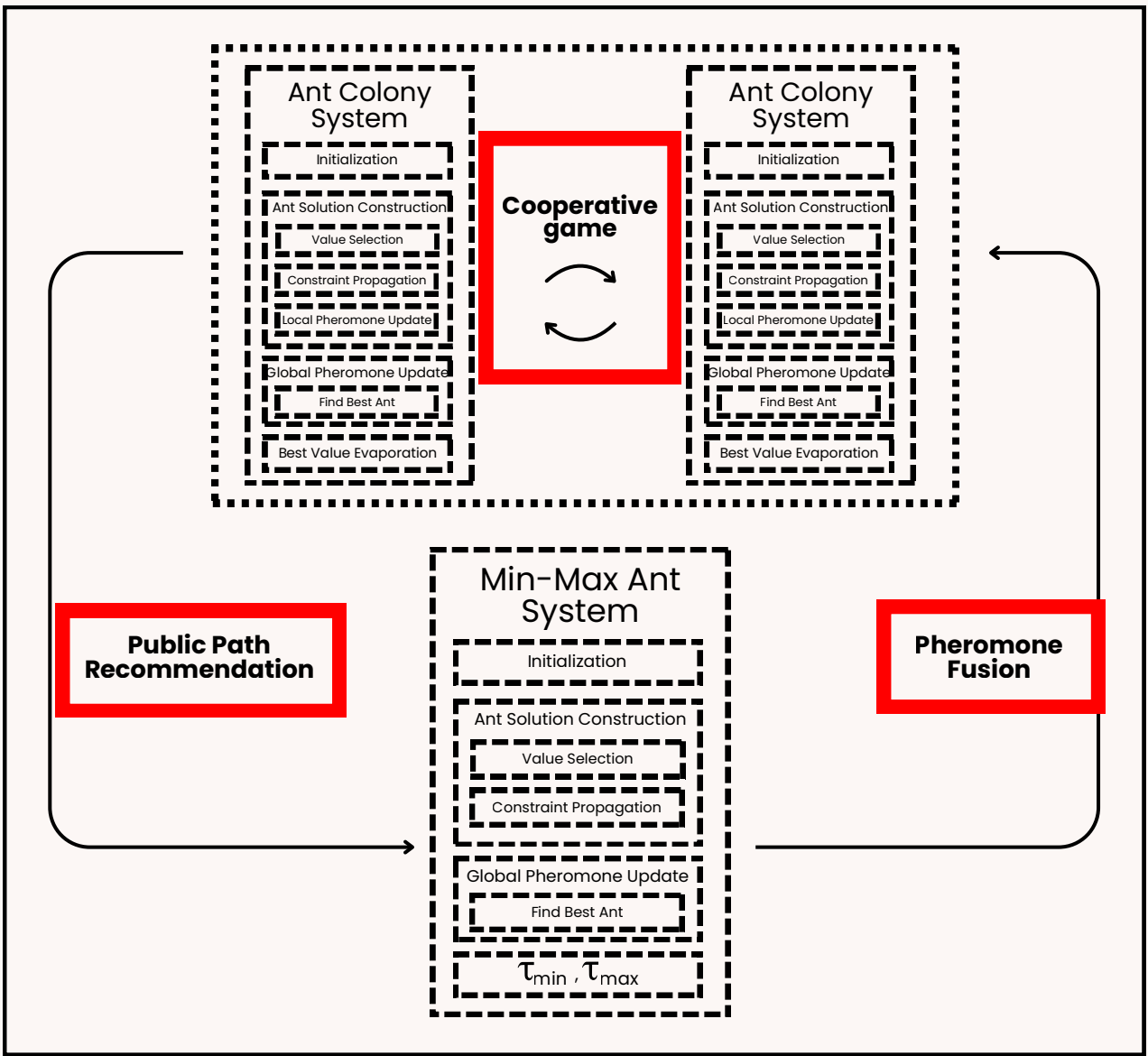
Methodology

CP-DCM-ACO



Dynamic Collaborative Mechanism

- consists of three mechanisms



Cooperative Game Theory

Pheromone Income

$$b = \sum_{i=1}^k \Delta \tau_{ACS}^i$$

Solution Quality

$$sol_i = \frac{numCells - f_{best}}{numCells - f_i}$$

Information Entropy

$$H(p_i) = \frac{E(p_i)}{E(p_{max})}$$

Pheromone Fusion

Weight Factor

$$\omega_i = \frac{E(P)_{ACS}^i}{E(P)_{ACS}^i + E(P)_{MMAS}}$$

Pheromone Update

$$ph_{ACS}^i \leftarrow (1 - \omega_i) \cdot ph_{ACS}^i + \omega_i \cdot ph_{MMAS}$$

Public Path Recommendation

Convergence Rate

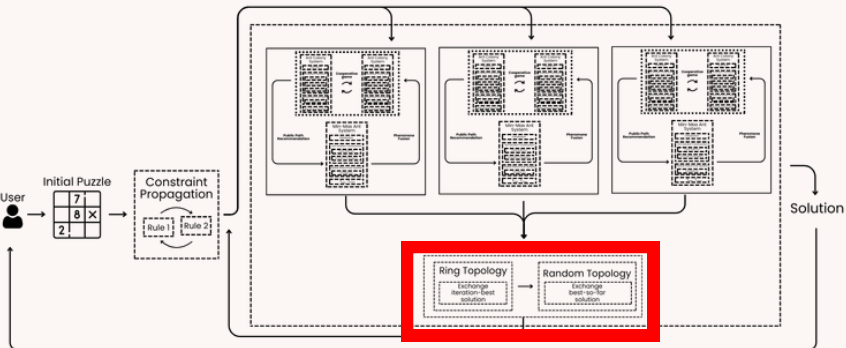
$$con_t = \frac{iter_{opt}}{iter_t}$$

Pheromone Value to Add

$$\tau_{public} = \frac{1}{N} \cdot e^{-iter}$$

Methodology

Threads Communication



The communication is triggered when the **current iteration = communication interval**

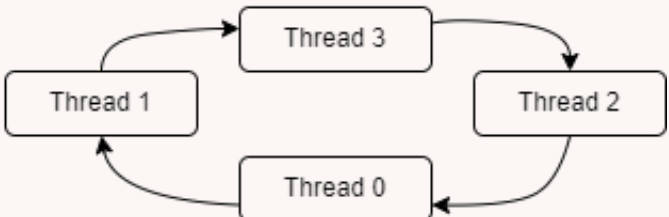
Random Sequence of Thread IDs [3, 2, 0, 1]



Ring Topology

Threads exchange their **iteration-best solution**.

Best solution from the current cycle.



Random-matching Topology

Threads exchange their **best-so-far solution**.

Best solution for all cycles.

Ring vs. Random Topology

MMAS Pheromones for Cell 2

	2	4	6	8
Cell 2	8.012	8	10	3

$$\Delta\tau_{ij} = \Delta\tau_{ij}^{(1)} + \Delta\tau_{ij}^{(2)} + \Delta\tau_{ij}^{(3)}$$
$$\tau_{ij} = (1 - \rho)\tau_{ij} + \rho\Delta\tau_{ij}$$
$$\tau_{i,j} = 0 + 9 + 0$$
$$\tau_{2,8} = (1 - 0.9)(3) + (0.9)9$$
$$\tau_{2,8} = 8.4$$

MMAS Pheromones for Cell 2

	2	4	6	8
Cell 2	8.8202	8	7.29	8.4

Assignments of values to cell 3 from different solutions from different threads.

Iteration-Best Solution of Thread 0
Thread 0 ($\Delta\tau = 8.91$)

1	2		7	9
---	---	--	---	---

Best-so-Far Solution of Thread 2
Thread 2 ($\Delta\tau = 8.1$)

1	6		7	9
---	---	--	---	---

Iteration-Best Solution of Thread 3
Thread 3 ($\Delta\tau = 9$)

1	8		7	9
---	---	--	---	---

Pheromone Contribution

Pheromone deposit from **own iteration-best solution**.

Exchanged Solutions

Pheromone deposit from **iteration-best solution of the ring neighbor**.

Each thread receives 2 solutions.

Pheromone deposit from the **best-so-far solution of the random neighbor**.

$$\Delta\tau_{ij} = \Delta\tau_{ij}^{(1)} + \Delta\tau_{ij}^{(2)} + \Delta\tau_{ij}^{(3)}$$

Results and Discussion

9×9 Sudoku: all methods solve reliably, but MCD requires substantially less time/iterations.

Size	#	Success Rate			Solution Time (s)			Iterations		
		CA	CD	MCD	CA	CD	MCD	CA	CD	MCD
3	1	100	100	100	0.27946±1.33186	0.03844±0.03515	0.01500±0.01164	348.52	55.02	18.34
3	2	100	100	100	0.09761±0.08400	0.06766±0.05918	0.03080±0.02257	106.65	86.84	34.85
3	3	100	100	100	0.19893±0.17658	0.17534±0.17667	0.06258±0.05401	239.15	249.09	77.56
3	4	100	100	100	0.07278±0.06155	0.04432±0.04317	0.01763±0.01461	90.35	64.72	21.95
3	5	100	100	100	0.06933±0.07450	0.05714±0.05644	0.01807±0.02666	85.01	82.63	22.22
3	6	100	100	100	0.11602±0.10370	0.11278±0.09926	0.03765±0.03712	132.10	155.12	45.01
3	7	100	100	100	0.08228±0.07224	0.06064±0.05312	0.02887±0.02163	101.41	88.67	35.95
3	8	100	100	100	0.04279±0.03319	0.03408±0.03501	0.01305±0.01147	51.29	48.46	15.38
3	9	100	100	100	0.10116±0.09460	0.10701±0.11034	0.04497±0.04327	119.70	151.96	55.05
3	10	100	100	100	0.08880±0.08230	0.05977±0.06851	0.02485±0.02452	115.58	91.95	32.39

CA: Single-threaded Single-Colony Solver (CP-ACS)

CD: Single-threaded Multi-Colony Solver (CP-DCM-ACO)

MCD: Multithreaded Multi-Colony Solver (MT-CP-DCM-ACO)

Results and Discussion

16×16 Sudoku: CD and MCD reach 100% success, with MCD using much less time and fewer iterations.

Size	#	Success Rate			Solution Time (s)			Iterations		
		CA	CD	MCD	CA	CD	MCD	CA	CD	MCD
4	1	74.00	100	100	6.20982±20.28314	0.04145±0.05942	0.01601±0.01370	1525.14	11.69	3.77
4	2	85.00	100	100	18.08683±28.34331	0.71115±1.72994	0.09718±0.17247	4163.73	200.11	23.86
4	3	83.00	100	100	2.89641±8.70555	0.30770±0.60110	0.05008±0.07103	571.89	74.97	10.73
4	4	87.00	100	100	1.73942±6.44938	0.04593±0.07579	0.01641±0.00974	480.17	15.05	4.34
4	5	79.00	100	100	0.45011±2.72285	0.06629±0.20343	0.02128±0.01122	108.78	18.38	4.89
4	6	95.00	100	100	1.86042±12.94886	0.02314±0.03614	0.01319±0.00503	417.65	6.18	2.92
4	7	86.00	100	100	3.28445±10.19821	1.22681±2.74470	0.15500±0.38457	706.95	303.85	34.37
4	8	95.00	100	100	6.97275±11.32015	2.51737±4.00437	0.53709±0.88221	1397.71	585.41	113.83
4	9	95.00	100	100	0.19367±1.18520	0.02628±0.03429	0.01666±0.00660	49.93	7.54	4.00
4	10	95.00	100	100	8.55567±13.92960	0.10995±0.20505	0.02948±0.02523	1775.49	28.05	6.54

CA: Single-threaded Single-Colony Solver (CP-ACS)

CD: Single-threaded Multi-Colony Solver (CP-DCM-ACO)

MCD: Multithreaded Multi-Colony Solver (MT-CP-DCM-ACO)

Results and Discussion

25×25 Sudoku: the difference in reliability becomes much clearer, with MCD reaching 83–100% success across the ten instances.

Size	#	Success Rate			Solution Time (s)			Iterations		
		CA	CD	MCD	CA	CD	MCD	CA	CD	MCD
5	1	23.00	85.00	97.00	3.74439±13.57122	19.36075±27.16000	16.81780±32.05523	269.26	1782.91	1448.79
5	2	32.00	75.00	95.00	17.54133±23.69841	33.27490±32.30204	14.83053±15.22324	785.84	1854.13	668.64
5	3	13.00	28.00	83.00	20.87791±32.39397	46.38122±38.37388	33.40157±28.57895	961.31	2486.86	1535.53
5	4	35.00	79.00	98.00	29.29770±29.70103	26.40158±27.47219	13.10972±15.33234	1184.60	1473.46	569.80
5	5	43.00	87.00	100.00	16.41774±22.18601	20.28063±21.22839	9.36144±10.09255	679.02	1157.68	424.63
5	6	35.00	91.00	99.00	12.32504±18.19699	25.52842±23.81864	14.15018±15.13670	601.57	1480.24	652.54
5	7	88.00	100.00	100.00	7.22602±14.82063	5.64181±5.76208	2.35472±1.64611	345.93	303.58	108.50
5	8	50.00	96.00	100.00	5.86453±16.35849	10.33280±20.46834	2.17646±8.74595	440.48	871.66	146.55
5	9	42.00	92.00	94.00	10.02994±23.35498	14.39664±22.60081	4.86837±14.27821	738.02	1180.67	319.94
5	10	54.00	84.00	100.00	15.87148±20.93051	26.30110±23.59125	17.07738±16.80106	805.41	1500.80	766.01

CA: Single-threaded Single-Colony Solver (CP-ACS)

CD: Single-threaded Multi-Colony Solver (CP-DCM-ACO)

MCD: Multithreaded Multi-Colony Solver (MT-CP-DCM-ACO)

Increasing Number of Threads

For 25×25 Instance 03, increasing the number of threads improved success rate and reduced solution time.

25 x 25 Sudoku Puzzle (Instance 03)				
Threads	Success (%)	Time (s)	Iterations	Idle Time (s)
1	35	32.90 ± 32.56	2027.03	0.00000
2	54	29.05 ± 27.33	1658.87	0.00238
4	85	26.18 ± 25.33	1328.13	0.00733
8	99	20.13 ± 20.32	776.82	0.01689
16	100	18.46 ± 19.27	432.00	0.20003

Conclusion



Multilevel collaboration becomes increasingly beneficial as Sudoku size and difficulty increase, with MCD maintaining 83–100% success across the 25×25 instances and substantially improving reliability over the baseline approaches. This helps preserve diversity, reduce stagnation, and improve reliability in large-scale Sudoku.

Recommendation

Future research could:

- Explore other communication topologies other than ring and random topologies.
- Adapt the proposed framework to other highly constrained problems such as university timetabling and job-shop scheduling.

References

- [1] H. Lloyd and M. Amos, "Solving sudoku with ant colony optimization," IEEE Transactions on Games, vol. 12, no. 3, pp. 302–313, 2020.
- [2] H. Simonis, "Sudoku as a constraint problem," in CP Workshop on modeling and reformulating Constraint Satisfaction Problems, vol. 12. Citeseer Sitges, Spain, 2005, pp. 13–27.
- [3] D. Eppstein, "Computational complexity of games and puzzles," in Proceedings of the International Conference on Fun with Algorithms. Elba Island, Italy, 2006, pp. 1–12.
- [4] T. Yato and T. Seta, "Complexity and completeness of finding another solution and its application to puzzles," in IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences, vol. 86, no. 5, 2003, pp. 1052–1060.
- [5] E. C. C. is Post-Doctoral, "Techniques for solving sudoku puzzles," arXiv preprint arXiv:1203.2295, 2012.
- [6] M. Dorigo, "Optimization, learning and natural algorithms," Ph.D. Dissertation, Politecnico di Milano, 1992.
- [7] P. Norvig, "Solving every sudoku puzzle," 2006, accessed: 2025-12-11. [Online]. Available: <http://norvig.com/sudoku.html>
- [8] M. Schottlender, "The effect of guess choices on the efficiency of a backtracking algorithm in a sudoku solver," in IEEE Long Island Systems, Applications and Technology (LISAT) Conference 2014, 2014, pp. 1–6.
- [9] A. Bhattarai, D. Uprety, P. Pathak, S. N. Shrestha, S. Narkarmi, and S. Sigdel, "A study of sudoku solving algorithms: Backtracking and heuristic," arXiv preprint arXiv:2507.09708, 2025.
- [10] J. F. Crook, "A pencil-and-paper algorithm for solving sudoku puzzles," Notices of the AMS, vol. 56, no. 4, pp. 460–468, 2009.
- [11] T. Mantere and J. Koljonen, "Solving, rating and generating sudoku puzzles with ga," in 2007 IEEE Congress on Evolutionary Computation. IEEE, 2007, pp. 1382–1389.
- [12] M. Dorigo and T. Stützle, Ant Colony Optimization. MIT press, 2004.
- [13] M. Pedemonte, E. Alba, and F. Luna, "Bit-parallel aco for the traveling salesman problem," Applied Intelligence, vol. 35, no. 3, pp. 346–357, 2011.
- [14] M. Randall and A. Lewis, "A parallel implementation of ant colony optimization," Journal of Parallel and Distributed Computing, vol. 62, no. 9, pp. 1421–1432, 2002.
- [15] Q. Yang and L. Fang, "Rmaco: A randomly matched parallel ant colony optimization," World Wide Web, vol. 19, no. 6, pp. 1009–1022, 2015.
- [16] Y. Mo, Z. Tang, and L. Zhao, "Multi-colony ant optimization with dynamic collaborative mechanism and cooperative game," Information Sciences, vol. 608, pp. 892–908, 2022.
- [17] Y. Manyam, S. Sundaram, and A. Sahay, "Sudoku solver algorithms: A comparative analysis," in 2024 First International Conference for Women in Computing (InCoWoCo). IEEE, 2024, pp. 1–9.
- [18] N. Pillay, "Finding solutions to sudoku puzzles using human intuitive heuristics," South African Computer Journal, vol. 49, no. 1, pp. 25–34, 2012.
- [19] D. E. Knuth, The Art of Computer Programming, Volume 4B: Combinatorial Algorithms, Part 2. Reading, Massachusetts: Addison-Wesley Professional, 2022.
- [20] R. Barták, "Constraint propagation and backtracking search," Proceedings of the CP Workshop on Constraint Propagation, 2005.

References

- [21] A. Moraglio, J. Togelius, and S. Lucas, "Product geometric crossover for the sudoku puzzle," in 2006 IEEE International Conference on Evolutionary Com putation. IEEE, 2006, pp. 470–476.
- [22] A. Moraglio and J. Togelius, "Geometric particle swarm optimization for the sudoku puzzle," in Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation, ser. GECCO '07. New York, NY, USA: Association for Computing Machinery, 2007, p. 118–125. [Online]. Available: <https://doi.org/10.1145/1276958.1276975>
- [23] J. A. Pacurib, G. M. M. Seno, and J. P. T. Yusiong, "Solving sudoku puzzles using improved artificial bee colony algorithm," in 2009 Fourth International Conference on Innovative Computing, Information and Control (ICICIC), 2009, pp. 885–888.
- [24] A. Kaur and S. Goyal, "A survey on the applications of bee colony optimization techniques," International Journal on Computer Science and Engineering, vol. 3, 08 2011.
- [25] C. Solnon, "Ant colony optimization for real-world constraint satisfaction prob lems," Applied Intelligence, vol. 17, no. 1, pp. 33–40, 2002.
- [26] G. Karatas Baydogmus, "Probability selection for solving sudoku with ant colony optimization algorithm," 07 2022, pp. 161–165.
- [27] M. Soh, B. N. Tsofack, and C. T. Djamegni, "Multi-ant colony heuristic approach to solving the traveling salesman problem," ARIMA, vol. 34, 2021.
- [28] H. Xue, P. Zhang, and L. Yang, "A multiple ant colonies optimization algo rithm based on immunity for solving tsp," in 2010 International Conference on Computer and Computational Intelligence (ICCCI). IEEE, 2010, pp. 289–293.
- [29] J. Zhao, X. You, Q. Duan, and S. Liu, "Multiple ant colony algorithm combining community relationship network," Arabian Journal for Science and Engineering, vol. 47, 02 2022.



A Multithreaded Multi-Colony Ant Colony Optimization Framework with Ring and Random Communication Topologies for Solving Large-Scale Sudoku

John Michael E. Jabuen*

Gian Myrl D. Renomeron

John Paul T. Yusiong

